Fórum Avaliativo

Entrega da Atividade Avaliativa

Olá alunos.

Este fórum avaliativo tem o objetivo de propor 4 temas baseados nas unidades de aprendizagem, focadas em estudos de caso, no qual o grupo de alunos possa refletir e dar suas respostas. Cada tema valerá até 1,0 ponto podendo o grupo receber no máximo a nota 4,0.

Importante: Para conseguir a nota máxima dada pelo tutor deste fórum, o aluno deve ter no mínimo 60% de Progressão neste componente, progressões abaixo de 60% receberão nota proporcional.

Números máximo de alunos por equipe: **6 alunos**

Data máxima de entrega: **29/11/2025** (pode enviar antes)

Formato de entrega: Texto simples com capa identificando a instituição, os integrantes, e o professor tutor: Me Eduardo Chagas Ferreira", seguindo com cada estudo de caso e sua solução. O formato de entrega é importante e poderá sofrer perda de pontos sem a formatação indicada.

Abraços e bons estudos,

prof/tutor Me Eduardo Chagas

Atividades Avaliativas (Estudos de Caso)

Estudo de Caso 1

Você é desenvolvedor e tem experiência com o desenvolvimento de *software* e POO. Atualmente, trabalha no setor de tecnologia da escola XYZ. Carlos, seu gerente, pediu para você atuar no projeto de melhorias com o objetivo de implementar um sistema de gestão que será utilizado pelo setor de matrículas e secretaria. O cadastro acontece com o uso de formulários impressos e preenchidos pelo próprio aluno.

Abaixo, em um quadro azul-claro, está o seguinte texto com trechos em destaque:

Para tanto, você deverá cumprir os seguintes requisitos:

É preciso permitir que a aplicação tenha campos de simples preenchimento para cadastro do aluno. As informações a serem cadastradas são:

- Nome do aluno
- Endereço residencial
- Telefone de contato
- Endereço de *e-mail*
- Matrícula

Na sequência, em outro quadro azul mais escuro, está o texto:

Para preencher os requisitos, você mapeou as principais tarefas a serem executadas:

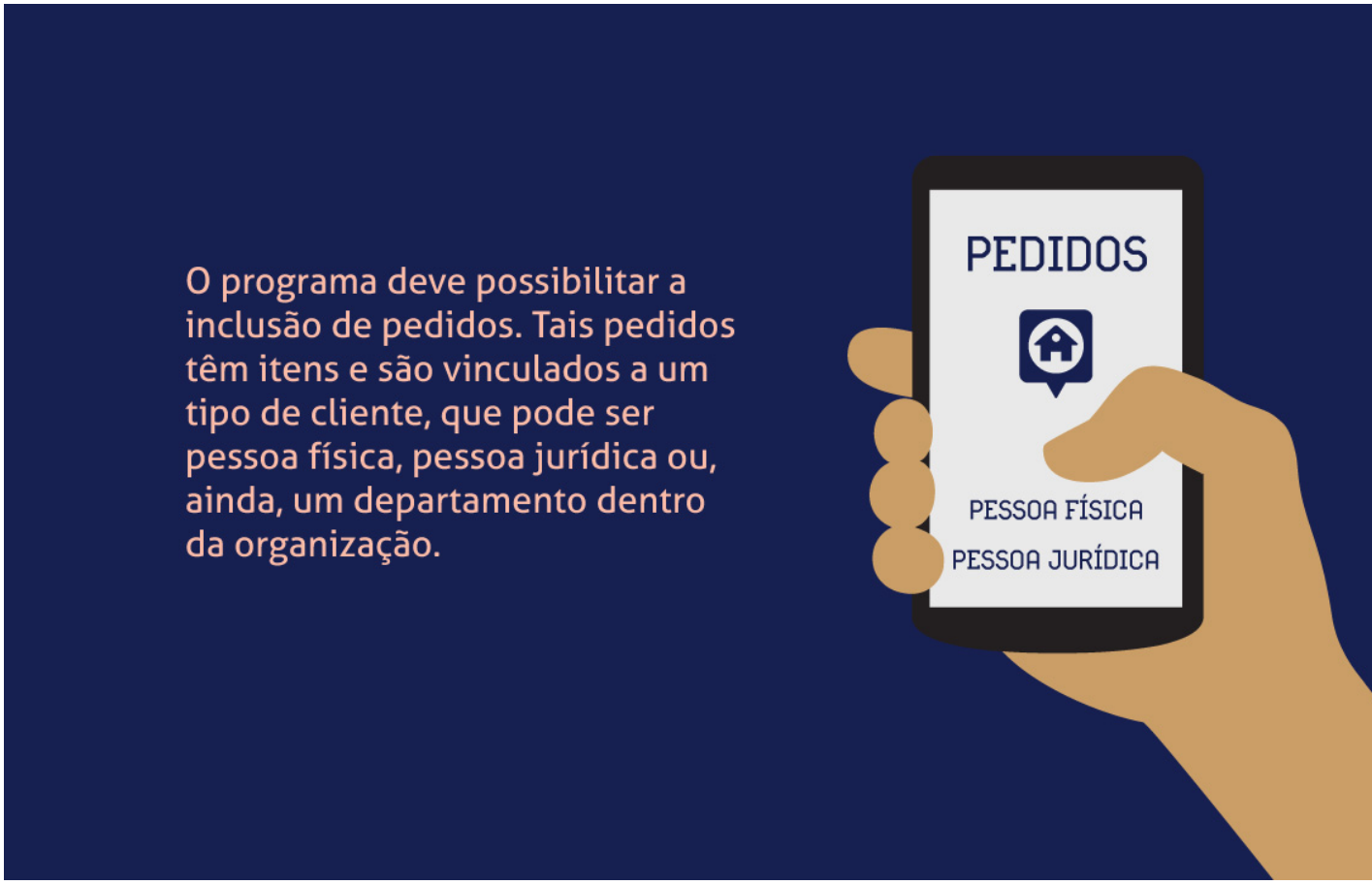
- Criar uma classe de definição que receba o nome do **Aluno**.
- Criar uma segunda classe denominada **Principal**.
- A partir da segunda classe criada, instanciar a classe Aluno.
- Relacionar valores ao atributo criado, adicionando alguns valores, e depois exibindo-os a fim de testar a funcionalidade do código.

Prints de tela devem ser criados demonstrando as codificações e o programa funcionando.

Estudo de Caso 2

As classes têm diferentes tipos de relacionamentos, que possibilitam especificar comportamentos e necessidades diferentes. Por exemplo, relacionamentos de associação (agregação ou composição) são tipos distintos e são utilizados para representar situações diferenciadas, assim como os relacionamentos de herança (generalização ou especialização) são utilizados para representar situações de heranças entre classes.

Imagine que você está participando da etapa de análise e projeto de um sistema que será construído em uma linguagem orientada a objetos. O escopo do sistema, definido até o momento, considera o seguinte:



Sem se preocupar com atributos e métodos, você deverá identificar as possíveis classes e seus relacionamentos, justificando a sua escolha.

Estudo de Caso 3

O principal conceito da programação orientada a objetos é que todo programa é uma coleção de objetos que se relacionam e apresentam comportamentos e atributos únicos.
Imagine que uma empresa contratou você para desenvolver um programa simples em uma linguagem orientada a objetos. O escopo desse programa é controlar os pedidos feitos em uma lanchonete.

A seguir, no Desafio, veja como os pedidos são feitos atualmente e como eles deverão ser realizados após o desenvolvimento do programa.

Com base nos requisitos estudados até aqui, elabore uma descrição das possíveis classes, métodos e atributos que seu programa deve apresentar para contemplar o escopo desse cenário. Os atributos e os métodos não precisam ser definidos, mas apenas identificados.

Estudo de Caso 4

A utilização de interfaces em linguagens de POO é um recurso muito benéfico em determinadas situações. Por meio do uso de interfaces, é possível determinar uma obrigação de definição de métodos por classes que implementem as interfaces, garantindo, assim, que, dentro de um contexto, as classes se comportem com uma similaridade esperada.

Com base nisso, imagine a seguinte situação:

Fazendo uso do conceito de interfaces, descreva em formato textual e apresente o modelo de classes:

a) escopo da interface chamada LoginPortal;

b) escopo de uma das classes que a implementa, chamada SistemaMarketing, que também tem o atributo codigoColaborador.

Estudo de Caso 5

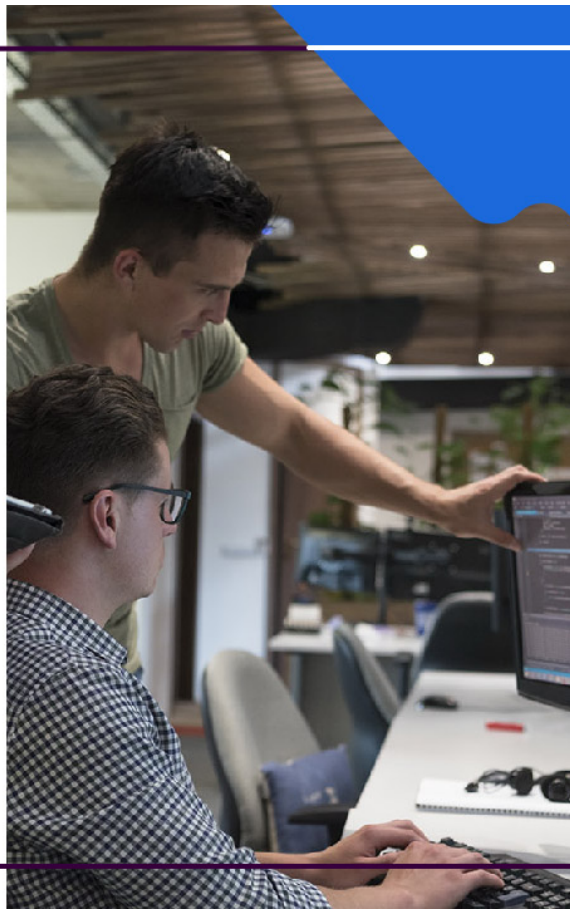
A matemática tem diversos números e sequências clássicas que são utilizadas em sistemas computacionais, como a sequência de Fibonacci. Trata-se de uma sequência de números inteiros, com os primeiros números sendo 0 e 1, e cada termo subsequente é definido pela soma dos dois números anteriores. Sendo assim, os primeiros dez números dessa sequência seriam 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, e assim sucessivamente.

Muito utilizada na arquitetura e nas artes, como, por exemplo, nas pirâmides, em quadros (p. ex., *Monalisa*) e em peças publicitárias, atualmente a sequência de Fibonacci é bastante implementada em aplicações de análise de mercados financeiros, estatística e teoria dos jogos. A sua formulação matemática simplificada indica uma função g tal que $g(n + 2) = g(n) + g(n + 1)$.

Considere a situação a seguir.

Você é desenvolvedor em uma empresa que foi contratada para desenvolver um sistema para análise financeira. Entre os conhecimentos que a sua equipe precisa adquirir, está a **sequência de Fibonacci.**

A lógica dessa sequência é simples: somar os dois últimos valores, a fim de encontrar o próximo da sequência, repetindo esse processo até que a quantidade de valores da sequência seja encontrada.



Considerando as características da sequência de Fibonacci e a necessidade de utilizar um comando de repetição, apresente um algoritmo escrito em Java que contenha os primeiros 50 números dessa sequência.

Estudo de Caso 6

Trabalhar com construtores e herança em orientação a objetos pode trazer muitas vantagens, como diminuir complexidade no desenvolvimento da aplicação. Entretanto, é importante ter bom conhecimento sobre o assunto a fim de utilizá-lo da melhor forma nos projetos.

Pensando nisso, analise o cenário a seguir.

Uma nova rede de farmácias foi aberta na cidade, e você foi contratado como analista/programador para atuar no projeto da fábrica de *software*. Sua tarefa é cuidar da aplicação de cadastro. No sistema, devem ser cadastrados os estabelecimentos e farmácias associadas à matriz, fornecedores de medicamentos e colaboradores.

Em reunião sobre as necessidades do estabelecimento e levantamento de requisitos entre o gerente e os representantes da associação, ficou definido que as seguintes informações precisam ser armazenadas:

— associados (ID, nome, logradouro, número, CEP, bairro, cidade, UF, telefone, CPF/CNPJ, situação e número de associado);

— colaborador (ID, nome, logradouro, número, CEP, bairro, cidade, UF, telefone, CPF/CNPJ, cargo ou função);

— fornecedores (ID, nome, logradouro, número, CEP, bairro, cidade, UF, telefone, CPF/CNPJ, nome fantasia e *website*).

Seu gerente lhe repassou algumas orientações para garantir que o projeto seja executado de maneira correta:

— criar as classes-modelo, utilizando técnicas com construtores e herança;

— utilizar uma linguagem de programação orientada a objeto (a recomendação foi o uso de Java);

— utilizar a IDE NetBeans na implementação do projeto para auxiliar na organização dele; de todo modo, você pode decidir entre outro IDE de sua preferência;

Faça prints de tela demonstrando a codificação criada nas classes